

Eustace the Pirate Monk — Specification

CS 2114 Project 1, Deliverable 2 — Group 87 (Marcos Salas, Aidan McIlvenni)

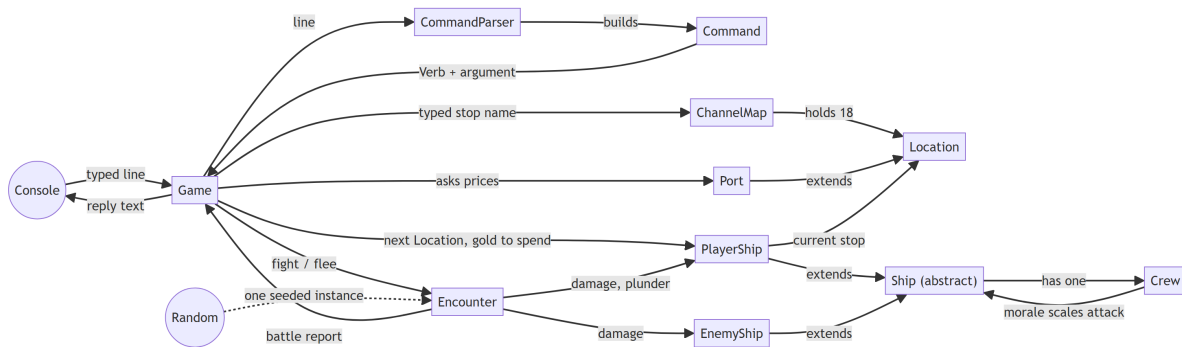
1. Class design

1. **Game**: runs the read–dispatch–print loop, decides which commands are allowed, and is the only class that prints the game (descriptions, actions, etc.). Holds `main` (necessary to run a java program).
2. **CommandParser**: splits one raw line into a `Command` exactly as typed, holds the table of verbs and their templates, and turns typed amounts into numbers.
3. **Command**: an immutable value, a verb word and its argument string.
4. **ChannelMap**: builds the fixed 18-stop map and calculates a stop from the name typed.
5. **Location**: one stop, with its key, name, kind, encounter chance and the stops one move away.
6. **Port**: a `Location` where the ship moors, with its nation and prices.
7. **Ship** (abstract): what both sides share, hull, cannons, armour, gold, rum and a `Crew`, plus the rules for dealing and taking damage.
8. **PlayerShip**: where the player is, notoriety, ports visited, and every purchase.
9. **EnemyShip**: a ship the player encounters, built from an `EnemyType` and the player's notoriety.
10. **EnemyType**: the base stats for one kind of enemy, with three shared instances, `MERCHANT`, `PIRATE` and `COAST_GUARD`, as static `final` constants.
11. **Crew**: head count, morale and greed, and the rules that move them.
12. **Encounter**: one meeting with an enemy, from the roll that creates it through the fight or the flight to the plunder.
13. **Describable** (interface): `String describe()`, implemented by `Location`, `Ship` and `Encounter`, so `Game` prints any of them the same way.

Fixed values are `String` constants: kinds `Location.PORT`, `SEA_LANE`, `HIGH_SEAS`; nations `Port.ENGLISH`, `FRENCH`, `PIRATE`. The twelve verbs `look`, `status`, `sail`, `repair`, `hire`, `buy`, `bonus`, `fight`, `flee`, `retire`, `help` and `quit` are the keys of `CommandParser.templates`. Both sides fight by the same rules, so `Encounter` calls `attackStrength()` and `takeDamage()` on any `Ship`. `Crew` is separate because morale scales attack on both sides and a mutiny ends the fight for either. Cannons and armour are levels up to 5 on one price list, so no port tracks stock; Sark sells rum at half price. Whether a command is allowed depends on two facts `dispatch` checks before running it: a fight is on while `encounter != null`, and the player is in port when their stop's kind is `Location.PORT`. Section 5 lists which verbs need which.

The map: seven ports (three English, three French, and Sark, the pirate home), eight sea lanes each joining two ports, and three high-seas stretches. `look` describes the current stop and lists its neighbours by the key to type, and `sail` moves to one of those neighbours: from Dover, `sail dover boulogne` enters the Dover–Boulogne lane, then `sail boulogne` reaches port. `find` looks the typed text up exactly as it is, so `sail Dover` or `sail dover-boulogne` finds nothing.

2. System diagram



Each arrow points the way the labelled thing travels. One turn: `Game` reads a line, `CommandParser` returns a `Command`, and `Game` checks the verb is allowed where the player is, runs it and prints the result.

3. Data & state

Where a field lists bounds, such as hull from 0 to `maxHull` or morale from 0 to 100, the methods that change it clamp to those bounds, so no caller can drive it outside.

- **Game**: `Scanner in`; `final ChannelMap map` and `PlayerShip player`; `Encounter encounter`, null unless a fight is on; `final Random random`, the only one in the program; `boolean running`; `static final int GOLD_TARGET = 1000 default`.
- **CommandParser**: `final HashMap<String, String> templates`, which maps each verb to its template, the form the command is typed in, such as `sail` to `sail <stop>`, filled in the constructor with the twelve verbs, so one lookup answers both "is this a verb" and "what is its template". **Command**: `final String verb`, the text before the line's first space, or the whole line when it has none; `final String argument`, the text after that space, "" when absent. Neither is trimmed, lower-cased or stripped of punctuation.

- ChannelMap: final HashMap<String, Location> stops, the 18 stops keyed by the stop's name in lower case with a space between words, such as dover boulogne; Port home, Sark, where the player starts and retires. ChannelMap.standard() builds the game's map: it puts the 18 stops into stops, keyed by the text the player types, with no array of stops; it then calls connect once per link, and connect puts each of the two stops into the other's neighbours array. It sets home, and nothing changes the map after it returns. sail looks the typed key up in stops.
- Location: final String key, name and kind; final int encounterPercent, 0 for a port, 35 for a sea lane and 60 for the high seas default; final Location[] neighbours of MAX_NEIGHBOURS = 4 slots and int neighbourCount, 2 to 4 once the map is built, where if A lists B then B lists A, and a stop never lists itself or a duplicate. No method hands the array out, so only connect can add a link, and neighbourList() gives the neighbours' keys in the order they were connected.
- Port adds final String nation and the default prices REPAIR_PRICE = 2, HIRE_PRICE = 15, RUM_PRICE = 4 and UPGRADE_PRICE = 100 gold per unit, with level n costing UPGRADE_PRICE × n.
- Ship: final String name; int hull, 0 to maxHull; final int maxHull; int cannons, 1 to 5; int armour, 0 to 5; int gold and int rum, never negative; final Crew crew. Subclasses change hull, cannons and armour only through setters, which are limited to the bounds above, so a PlayerShip repair or upgrade can never push hull past maxHull or a level past 5.
- PlayerShip adds Location location, never null; int notoriety; int portsVisited; static final int MAX_LEVEL = 5. It starts default at Sark with hull 100, crew 20 at morale 70, cannons 1, armour 1, gold 200 and rum 30.
- EnemyShip adds final EnemyType type. Its constructor takes the type's base numbers and adds an amount set by the player's notoriety: 2 hull per point, 1 crew per 2 points (capped at 40), 1 cannon and 1 armour per 10 points (each capped at 5), and 5 gold per point. Rum starts equal to the crew count and morale comes from the type, so at notoriety 10 a merchant has hull 60, crew 13, cannons 2, armour 1 and gold 200.
- EnemyType: final String name, and final int hull, crew, cannons, armour, gold, morale and gain. The three default constants, in that order: MERCHANT 40, 8, 1, 0, 150, 30, 1; PIRATE 60, 15, 2, 1, 100, 60, 2; COAST_GUARD 80, 20, 3, 2, 50, 80, 3.
- Crew: int count, 0 to MAX_COUNT = 40; int morale and int greed, both 0 to 100, where greed starts at 0 and only the player's crew ever gains it. Default steps: each man lost costs 5 morale, each bottle of rum deficit costs MORALE_PER_DRY_BOTTLE = 5, a win pays 10 back, a flee costs 15, and every 20 gold of plunder adds 1 greed. A bonus of g gold to a crew of c men lowers greed by $g \div c$ and raises morale by half of $g \div c$, both rounded down, so 100 gold to 20 men takes 5 off greed and adds 2 to morale.
- Encounter: final PlayerShip player, EnemyShip enemy and Random random; FLEE_DAMAGE = 10 and ROLL_RANGE = 6 default.

How the numbers move:

1. moraleFactor() is $0.5 + \text{morale} \div 200.0$, running from 0.5 at morale 0 to 1.0 at morale 100. attackStrength() multiplies that factor by 5 points per cannon plus 1 point per two crew, throws away the fraction and never gives less than 1. A fresh player ship has 1 cannon and 20 crew at morale 70, so $(5 + 10) \times 0.85 = 12.75$, which floors to 12.
2. takeDamage(raw) works out what gets through the armour, $\text{raw} - 2 \times \text{armour}$, at least 1 when raw is above 0. The hull drops by that number and the crew loses one man per 5 points of it, rounded down: 12 raw against armour 1 gets 10 through, so hull drops 10 and 2 men die.
3. isDefeated() is true at hull 0, at crew 0, or when crew.mutinied() is true, which happens at morale 0 or greed 100. The player's crew mutinying ends the run; an enemy crew mutinying stops its fire, printed as a surrender.
4. A fight round: the player fires attackStrength() + random.nextInt(ROLL_RANGE); if the enemy still stands it fires back the same way, and rounds repeat until one is defeated. A win moves the enemy's gold and rum across, adds the type's gain to notoriety, and calls crew.onWin(gold).
5. Each sail into a sea stop: the crew wants one bottle per ten men, a part-full ten rounding up, so 20 men want 2 and 21 want 3. It drinks what the hold has, and the rum deficit, bottles wanted minus bottles drunk, costs MORALE_PER_DRY_BOTTLE per bottle: 20 men with 1 bottle aboard drink it and lose 5 morale, and with an empty hold they lose 10. Then roll draws nextInt(100) against the stop's chance, and a second draw picks the type: in a sea lane below 70 is a merchant, else the coast guard; on the high seas below 50 is a pirate, else a merchant.
6. A purchase buys the smallest of three numbers: the amount typed, how many units the gold covers ($\text{gold} \div \text{price}$, rounded down), and the room left before hull, crew or a level reaches its cap. It then charges that many units times the price, which the gold always covers.

4. Method signatures

"Refuses" means the method returns 0 or false and changes nothing, and dispatch turns that into the section 5 message. IAE means it throws IllegalArgumentException.

Game: Game(Scanner in, ChannelMap map, Random random); static void main(String[] args) builds a game on System.in, ChannelMap.standard() and new Random() and runs it; void run() prints the opening scene and loops read, parse, dispatch, print while running is true; String dispatch(Command c) applies one command and returns the text to print, and when the command sinks the ship, mutinies the crew, retires the player or confirms quit, it sets running to false and ends its text with how the run ended, the final gold and the ports visited; boolean isRunning(); PlayerShip getPlayer(); Encounter getEncounter().

CommandParser: CommandParser() fills the verb table; Command parse(String line) splits the line at its first space and returns a Command holding the two parts unchanged; boolean isVerb(String word); String templateOf(String verb) gives the template or null; String allTemplates() joins every template for help; static int parseCount(String text) returns an int of 0 or more, or throws NumberFormatException for anything else.

Command: Command(String verb, String argument), IAE on a null verb; String getVerb(); String getArgument(); String getWord(int i) gives the i-th argument word or ""; String toString() gives "sail dover".

ChannelMap: static ChannelMap standard() adds the 18 stops and calls connect for each of the 25 links; void add(Location stop) puts a stop into stops, IAE on a duplicate key; void connect(String keyA, String keyB) looks both keys up in stops and calls Location.connect, IAE on an unknown key; Location find(String name) gives the stop or null; Port getHome().

Location implements Describable: Location(String key, String name, String kind, int encounterPercent); void connect(Location other) adds each to the other once, IAE on itself or when either already has 4; boolean isNextTo(Location other); String neighbourList() gives the neighbours' keys joined by commas, such as "barfleur sark, west channel"; getters; String describe() gives name, kind and neighbourList(). Port: Port(String key, String name, String nation) passes kind PORT and chance 0 upward; String getNation(); int rumPrice(), halved at Sark; int upgradePrice(int nextLevel); describe() adds the prices.

Ship implements Describable: Ship(String name, int maxHull, int cannons, int armour, Crew crew, int gold, int rum); int attackStrength(); int takeDamage(int raw) returns hull lost; boolean isDefeated(); void addGold(int); void spendGold(int), IAE beyond what is held; void addRum(int); int takeRum(int) returns bottles taken, capped at the hold; Crew getCrew(); a getter per field; abstract String describe().

PlayerShip: PlayerShip(String name, Port home); void moveTo(Location d), IAE unless d is a neighbour, counting a port arrival; int repair(int points, int pricePerPoint), int hire(int men, int pricePerMan) and int buyRum(int bottles, int pricePerBottle) each buy by the rule in section 3 and return the units bought; boolean upgradeCannons(int price) and boolean upgradeArmour(int price) refuse at MAX_LEVEL or without the gold; boolean payBonus(int gold) refuses beyond what is held, else hands the gold to the crew; void addNotoriety(int); getters; describe() is the status report.

EnemyShip: EnemyShip(EnemyType type, int notoriety), IAE on null or a negative; EnemyType getType(); describe() is what the lookout sees. EnemyType: a constructor taking the eight values in section 3 and a getter per field; the three constants are its only instances.

Crew: Crew(int count, int morale) caps both; int lose(int men) and int hire(int men) return the number removed or added after capping; double moraleFactor(); int drink(int bottlesAvailable) returns bottles drunk and charges morale for the rum deficit; void onWin(int plunder); void onFlee(); void receiveBonus(int gold); boolean mutinied(); getters.

Encounter implements Describable. Every random decision in a meeting is a draw from the Random passed in, the one main creates, so a test that passes a FixedRandom fixes each outcome. Encounter(PlayerShip player, EnemyShip enemy, Random random) keeps random for the damage rolls; static Encounter roll(Location where, PlayerShip player, Random random) draws whether an encounter happens and which EnemyType appears, builds new EnemyShip(type, player.getNotoriety()) and returns a new encounter, or null when none happens; String fight() resolves the whole battle, adding a random draw to every shot, and returns the narration; String flee() calls player.takeDamage(FLEE_DAMAGE), then crew.onFlee(), which takes 15 morale, and returns the text to print; EnemyShip getEnemy() returns the ship being fought, so Game and the tests can read its type, hull and crew; String describe() returns what look prints during the meeting which is the enemy ship's own describe().

5. Where validation lives

dispatch checks where the player is first: fight and flee need encounter != null, sail is refused while it is, and repair, hire, buy and bonus also need the player's stop to be a port.

1. Blank line: parse gives verb "" and dispatch returns "".
2. With an Unknown verb (ex: 'sial') isVerb is false, so dispatch prints "I don't understand 'sial'. Type help."
3. Missing argument: dispatch sees an empty argument and prints that verb's template (ex: sail).
4. Extra words: find("dover now please") gives null and parseCount("5 men") throws, so sail dover now please gets "no such place" and hire 5 men replies 'you can only hire in integers'.
6. Case, spacing, quotes, punctuation, invisible characters: parse keeps the text as typed, so SAIL dover fails isVerb and gets the case 2 reply, sail dover gives verb "" and the case 1 reply, and sail Dover. gets "no such place".
7. A real stop that is not adjacent: find succeeds, isNextTo is false, so dispatch prints the current stop's neighbourList().
8. sail help: the verb is the first word only, find("help") gives null, and the reply is "no such place".
- 9, 11. A negative amount, "ten" or hiring "10.1": parseCount throws NumberFormatException and dispatch asks for a whole number of 0 or more.
10. Zero: parseCount gives 0, the purchase returns 0, the reply is "bought 0".
12. Overflow: PlayerShip divides gold by price before multiplying, so hire 2147483647 by a fresh player, who starts with 200 gold, buys $200 \div 15 = 13$ men for 195 gold.
- 14, 15. A verb in the wrong state: the dispatch check can print "nothing to fight" or "you must be in port".
16. An unknown item: dispatch sees a word it doesn't know and prints what words it does know.
17. retire away from home or short of gold: dispatch checks the stop against home and the gold against GOLD_TARGET, then prints the gold still needed.
- 19, 20. Spamming look or status, and sail <current stop>: those branches never call roll or drink, and a target equal to the current stop gets "you are already there".

22. An upgrade at the maximum: the upgrade methods check the level before the gold, so the gold is untouched.
23. A 100,000-character line, and injection: `isVerb` looks the first word up in the fixed verb table, and `find` and `buy` compare against fixed keys and three item words, so typed text is not misinterpreted.
24. Input ends: `run` checks `hasNextLine()` before every read, prints "input ended" and sets `running` to false.
26. Accents and emoji: `find` gives null and no item word matches, prints that it doesn't understand that word.

6. Test plan

Game:

- `Game(...)` and `run`: a new game starts at Sark with no encounter and `isRunning()` true. Given a Scanner holding the lines `look`, `status`, `quit` and `yes`, `run` prints "Sark" and "Hull 100/100", then ends the game, so `isRunning()` is false. Bad: a null map throws IAE, and given a blank line, then `sail`, then no more lines, `run` prints the unknown-command reply and "input ended" without throwing.
- `dispatch`, normal: `sail barfleur sark` reaches the lane with `rum` 28; `hire 5` gives crew 25 and gold 125; with `FixedRandom(0)`, a `Random` that always rolls 0, `fight` at the lane ends with gold 350 and no encounter; `retire` on 1000 gold ends the game; `look` prints the stop name, status "Morale 70", and `help` every template.

CommandParser and Command:

- `parse`, `isVerb`, `templateOf`, `allTemplates` and `parseCount`: parsing `sail dover boulogne` gives verb "sail" and argument "dover boulogne"; `templateOf("sail")` gives `sail <stop>`; `allTemplates()` contains all twelve templates; `parseCount("25")` gives 25 and `parseCount("0")` gives 0. Bad: parsing `SAIL "Dover"` gives verb "SAIL" and argument "Dover". unchanged; a blank line and `sail dover` both give verb ""; `isVerb` is false for `sial`; `parseCount` throws `NumberFormatException` for "-50", "ten", "10.1" and "999999999999".

ChannelMap, Location and Port:

- `ChannelMap.standard()`: a map of 18 stops with home sark, every link recorded on both stops.
- `ChannelMap.add`: a duplicate key throws IAE.
- `ChannelMap.connect`: `connect("dover", "atlantis")` throws IAE.
- `ChannelMap.find`: `dover boulogne` gives the lane; `dover now please`, `dover-boulogne`, `atlantis`, `help` and `dövé` give null.
- `Location(...)`: a lane built with chance 35 reports 35.
- `Location.connect`: afterwards each stop lists the other; connecting a pair twice lists each once; `a.connect(a)` and a fifth link throw IAE.
- `Location.isNextTo`: true for a connected stop, false for any other.
- `Location.neighbourList`: Sark gives "barfleur sark, west channel".
- `Location.describe`: Sark's text contains its neighbour list.
- `Port.rumPrice`: 4 at Dover, 2 at Sark.
- `Port.upgradePrice`: `upgradePrice(2)` gives 200, what the port charges to upgrade canons and armour to level 2. The rule is: level * `UPGRADE_PRICE`
- `Port.describe`: Dover's text contains its prices.

Ship, PlayerShip and EnemyShip:

- `Ship(...)`: a test subclass with `maxHull` 40 starts at hull 40; `maxHull` 0 or a null crew throws IAE.
- `Ship.attackStrength`: 12 for a fresh player, 7 with the crew at morale 0.
- `Ship.isDefeated`: false for a fresh player, true after `setHull(0)`.
- `Ship.takeDamage`: 12 against a fresh player returns 10, leaving hull 90, crew 18 and morale 60; 500 returns 100, leaving hull 0; a negative throws IAE.
- `Ship.addGold` and `spendGold`: 50 gives 250 or 150; `spendGold(500)` throws IAE with gold still 200; a negative throws IAE.
- `Ship.addRum` and `takeRum`: `addRum(10)` gives 40; `takeRum(5)` returns 5; `takeRum(50)` returns 30 and empties the hold; a negative throws IAE.
- `PlayerShip(...)`: starts at Sark with notoriety 0; a null home throws IAE.
- `PlayerShip.moveTo`: sailing to the lane and back counts 1 port visited; Sark to Dover throws IAE.
- `PlayerShip.addNotoriety`: 2 gives 2; a negative throws IAE.
- `PlayerShip.describe`: the text holds every field.
- `PlayerShip.upgradeCannons` and `upgradeArmour`: `upgradeCannons(200)` at level 1 returns true, leaving level 2 and gold 0; at level 5 or with 50 gold, either returns false.
- `PlayerShip.payBonus`: `payBonus(100)` after `onWin(600)` gives greed 25 and morale 82; `payBonus(500)` returns false.
- `EnemyType` getters: `MERCHANT.getGain()` gives 1 and `COAST_GUARD.getGain()` gives 3.

Crew and Encounter:

- `Crew.drink`: a crew of 20 at morale 70 drinks 2 of 30 bottles and keeps morale 70; with 1 bottle it drinks 1 and drops to 65; with 0 it drops to 60; a negative throws IAE.
- `Crew.onWin`: `onWin(150)` gives morale 80 and greed 7; `onWin(5000)` caps greed at 100.
- `Crew.onFlee`: morale 70 drops to 55.
- `Crew.receiveBonus`: 100 gold after `onWin(600)` gives greed 25 and morale 82.
- `Encounter(...)`: a null enemy throws IAE.
- `Encounter.roll`: a sea lane with `FixedRandom(0)` gives a merchant encounter; a port, or the high seas with `FixedRandom(99)`, gives null.
- `Encounter.describe`: the merchant encounter's text names a merchant.

- `Encounter.fight`: against that merchant, the enemy surrenders in round 3, leaving hull 97, gold 350, rum 38, morale 80 and greed 7; after `setHull(5)`, a fight against (`PIRATE`, `0`) is lost in round 1 with gold still 200.
- `Encounter.flee`: a fresh player is left with hull 92, crew 19 and morale 50; after `setHull(5)`, the hull is emptied.

7. Division of work

Aidan owns `CommandParser`, `Command`, `Ship`, `PlayerShip`, `EnemyShip`, `EnemyType`, `Crew` and `Encounter`; Marcos owns `Game`, `ChannelMap`, `Location`, `Port` and `Describable`, sets the *default* numbers, submits to Web-CAT, and reworks the classes toward Aidan's suggestion that combat feel "gambly". On day one we write every class with its section 4 signatures and stub bodies, so the project compiles and each of us edits only our own files. Marcos merges each class into main with its test class at the Tuesday 7 pm meeting, the map, look, status, sail, repair, hire, buy, help and quit in week 1, then fights, the crew rules, bonus, retire, mutiny and sinking in week 2, and the merged build must pass Web-CAT with no style warnings. If time runs short we cut greed and bonus first, then rum in favour of a fixed morale drop per sea move, then the three high-seas stops, each cut leaving every other class unchanged.

8. Revised scope

3. Crew morale, greed, and rum went from being a stretch goal to an integral .
4. An abstract `Ship` shared by both sides, with enemies scaled by `EnemyType` and notoriety: one fight routine, and the difficulty scaling promised in `DESIGN.md` §1 returns. Marcos asked for a shared base; the GenAI draft chose inheritance.
5. Cannon and armour levels on one price list in place of weapon items with per-port stock, removing an `Item` class and seven stock lists. GenAI review, after Marcos set a two-week budget.
6. Overflow (case 12) handled by dividing before multiplying, refusals as return values with `dispatch` writing every message, and a `parse` that never throws, so one place makes all player-facing text. Aidan's draft; the division from GenAI review.
7. Commands restricted by two checks in `dispatch`, a fight on and the player in port, closing the §5 unknown about restricting commands by mode. Aidan's mode idea plus `DESIGN.md`'s "state machine" note; Marcos replaced a `mode()` method with the two checks.
8. One `Random` injected through `Game`, with `Encounter` fully constructible, closing the §5 unknown about forcing a fight in `JUnit`. GenAI review of §5 in both drafts.
9. No enums: kinds and nations are `String` constants, verbs are keys in a `HashMap` verb table, and `EnemyType` is a class with three static `final` instances, because the course has covered `HashMap`, interfaces and generics but not enums. Marcos, after GenAI review proposed enums.
10. No input clean-up: `parse` and `find` take text exactly as typed, since `help` shows every template and `look` every stop key, so `DESIGN.md` §4 cases 4–6 and 25 get the blank-line, unknown-command or "no such place" reply in place of being corrected. Marcos, to simplify parsing.